CampusLoop: A Campus Rental Prototype

Rujal Sharma (C-25)
Dept. of AI & Data Science
Dr. D. Y. Patil Institute of Technology, Pimpri, Pune
India

Sudhanshu Pendalwar (C-46)
Dept. of AI & Data Science
Dr. D. Y. Patil Institute of Technology, Pimpri, Pune
India

Shlok Mishra (C-60)
Dept. of AI & Data Science
Dr. D. Y. Patil Institute of Technology, Pimpri, Pune
India

Vansh Jain (C-61)
Dept. of AI & Data Science
Dr. D. Y. Patil Institute of Technology, Pimpri, Pune
India

***Abstract*— Students sometimes need equipment for a short period but have no structured way to record a peer rental. CampusLoop is a responsive browser-based prototype for exploring this interaction at one college. It provides sample listings, search and filters, item details, date requests, owner decisions, return confirmation, and ratings. A client-side state module checks ownership, date limits, overlapping active requests, and actor-specific status transitions. The application stores demo changes in localStorage on the current browser. Five automated logic tests passed in the inspected repository. These results establish functional behavior for selected scenarios only; they do not measure usability, adoption, environmental benefit, or production security. No real accounts, shared database, payment, deposit, or escrow service is implemented.**

Index Terms*— campus sharing, peer-to-peer rental, prototype, state transitions, browser-local storage

I. INTRODUCTION

Students may need a calculator, drafter, camera, textbook, or cycle only briefly. Buying each item can be expensive, while informal borrowing through messaging does not consistently record the owner, dates, price, and return state. CampusLoop investigates how these steps can be represented in a clear campus-specific interface. The present work is a software prototype, not an operational rental service.

The design objective was to let a presenter demonstrate both sides of a rental in one browser. A visitor can switch among four sample second-year AI and Data Science profiles, browse or list items, request dates, and act as the owner on the same saved state. This deliberate constraint supports walkthroughs but cannot synchronize different students or devices.

II. RELATED WORK

Belk describes online sharing and collaborative consumption as access to resources without conventional ownership [1]. Hamari et al. report economic and sustainability motivations for participation in collaborative consumption [2]. These studies frame the general concept; neither validates the CampusLoop interface. The United Nations Sustainable Development Goal 12 includes reuse in its waste-reduction target [3]. A local rental system could support reuse, but this prototype has not measured displaced purchases or waste.

CampusLoop therefore addresses a narrower design question: can a browser-local demonstration show a coherent borrower-owner workflow while rejecting inconsistent requests? Its evaluation is limited to deterministic functional checks in the source repository.

III. DESIGN AND SYSTEM BOUNDARY

A. User journey

A borrower opens the catalog, searches or filters sample listings, reviews condition and daily rate, selects start and end dates, accepts the displayed sharing terms, and sends a request. The owner profile can confirm or decline. After a confirmed rental, the borrower can request a return, the owner can mark it completed, and the borrower can assign one rating. The interface distinguishes borrowing from lending and exposes a reset control for demonstrations.

B. Architecture

React components render the screens; Vite is the development and build tool. A marketplace module in src/lib/marketplace.js contains the demo state and validation logic. Initial listings come from src/data.json. The browser writes the state to localStorage. No server API, account service, identity verification, shared database, or payment processing exists.

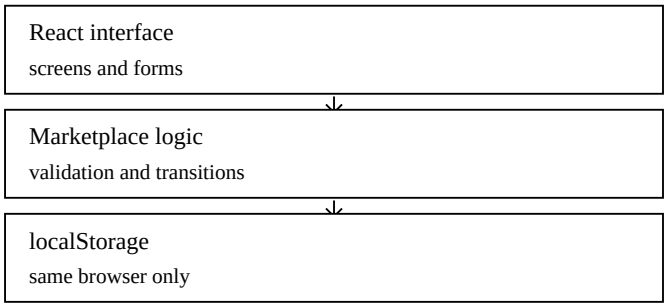


Fig. 1. Current prototype architecture. There is no server or shared database.

C. Interface decisions

The top navigation separates discovery, personal listings, borrowing and lending into named destinations, with a compact menu on smaller screens. Catalog cards place category, condition, owner and daily price together, while the detail screen exposes the item description and a single request action. The request form places date fields beside an estimated rent and states that no money is collected. These choices are intended to reduce ambiguity during a walkthrough; their effect on comprehension remains untested with participants.

D. Local state lifecycle

The seed function creates 12 sample items and three example bookings relative to the current date. The browser attempts to load saved JSON at startup, accepts supported state versions, and falls back to a fresh seed when stored data are damaged or incompatible. A migration step updates owner display names when older demo state is found. These measures preserve a repeatable presentation while allowing a user to try interactions. They do not resolve simultaneous requests on different devices.

IV. REQUEST STATE MODEL

Each request records an item, renter, date range, inclusive day count, estimated rent, status, and creation time. The estimate equals the inclusive number of days multiplied by the item daily rate. This is informational: the application collects no money. The state module rejects a missing or paused item, an unknown demo renter, a request for an item owned by the renter, a past start date, a reversed or more-than-30-day period, and an overlap with a Pending, Confirmed, or Return requested booking for the same item.

A. Overlap rule

Two closed date intervals overlap when requested_start <= existing_end and requested_end >= existing_start. Completed, Declined, and Cancelled requests do not block a new request. Inclusive dates make a one-day request valid when start and end match.

B. Actor-gated transitions

Current status	Next status	Actor
Pending	Confirmed or Declined	Owner
Pending	Cancelled	Renter
Confirmed	Return requested	Renter
Confirmed	Cancelled	Owner
Return requested	Completed	Owner

A completed request can be rated once by its renter with an integer score from 1 to 5. A listing cannot be paused while it has an active request. These checks maintain a consistent demonstration state, but browser-local enforcement is not a security boundary for real transactions.

The transition check computes the actor role from the booking renter identifier and the item owner identifier, rather than trusting a status button alone. It then compares the requested next status with a table of allowed transitions. The same module returns a new state object instead of mutating the current one in place, so React can render the changed view after an action. This is a local consistency rule, not proof of identity.

V. IMPLEMENTATION AND FUNCTIONAL CHECKS

The catalog groups sample items into Electronics, Lab, Books, and Cycles. Visitors can search, filter, and sort. Listing creation validates title and description lengths, category, condition, daily price, and approximate item value. New listings receive a category illustration rather than a user-uploaded photograph. Profile switching changes the perspective shown in My rentals and My listings.

We ran the repository's Node test suite on 13 October 2026. It reported five tests passed, zero failed. Table I lists the tested behavior. The test suite does not include a multi-user network test or measured user study.

Tested behavior	Observed result
Profile data and saved-state migration	Pass
Inclusive duration and reversed range	Pass
Rent estimate, overlap, own-item guard	Pass
Owner-only transitions and one-time rating	Pass
Listing validation and active-request pause guard	Pass

The visual captures in Figs. 2-5 were taken from the running demo. They document what a presenter sees, not real student transactions.

A. Test protocol

The checked suite uses Node's built-in test runner and assertions against the exported marketplace functions. Each test constructs the seeded state or a derived state, invokes a user action, and inspects the returned state or expected error. A future-date helper keeps booking cases independent of the day the suite is run. The tests do not exercise browser rendering; the screenshots provide a separate visual record of the interface.

B. Scope of evidence

A passing unit test confirms the specified rule on its test inputs. It cannot show that every edge case is handled, that a real student understands the interface, or that two devices agree on availability. The catalog and four profiles are sample data. The project has not collected survey responses or live transaction records, so no satisfaction, savings, or environmental outcome is reported.

C. Worked request trace

For the seeded calculator priced at INR 20 per day, the test constructs a three-day request and checks an estimated rent of INR 60 and an initial Pending status. It then attempts a second request for the same item whose dates intersect the first request. The second attempt raises an overlap error. A separate attempt by the item owner raises an own-item error. This trace demonstrates the relationship between the price calculation, interval rule, and identity check in one isolated state; it does not simulate a network race or transfer of the item.

VI. DISCUSSION AND LIMITATIONS

The prototype makes the core journey inspectable from both roles and keeps the absence of payments explicit. Its main limitation is that all participants are selectable sample identities in one browser. localStorage state is neither shared nor tamper-resistant. Consequently, the current implementation cannot establish trust, ownership, availability across devices, payment safety, or dispute resolution. A real deployment would require an authenticated backend, shared persistence, moderation, and institutional policy decisions.

The five passing tests support selected logic rules only. They do not establish task completion rates, accessibility, response time under load, economic viability, or environmental impact. A later study should recruit students, define tasks, observe completion and errors, and examine whether real listings and handoffs occur.

A. Threats to validity

The demonstration setting has no independent users: a presenter can switch both roles. This removes the uncertainty of a real owner response and makes observed task flow easier than deployment. Browser-local persistence can also be reset or altered by the same user. These properties are useful for reproducible classroom presentation but limit conclusions about trust, safety, or adoption.

B. Deployment prerequisites

A production design would require identity and authorization checks on a server, transaction-safe reservation updates to avoid races, shared item and booking records, and a documented procedure for handoff, complaints, and data retention. Payment or deposit handling would require a separate institutional and legal decision. None of these prerequisites should be inferred from the interface of the current prototype.

VII. REPRODUCIBILITY AND NEXT EVALUATION

The prototype can be run from the repository with a recent supported Node.js version using npm ci and npm run dev. The deterministic marketplace checks run with npm test. The initial state is built from the sample catalog and four named profiles; Reset demo restores it. Screenshots should be interpreted with their capture date because sample booking dates are generated relative to the current day.

The next evaluation should separate interface usability from marketplace viability. A usability study could ask participants to locate a listed item, interpret its condition and estimated cost, send a date request, switch to an owner view, and explain the return state. Time on task, completion, errors, and comprehension of the no-payment notice could be recorded. A later deployment pilot would require independent authenticated users and shared storage before measuring actual request acceptance or item reuse. Neither study has yet been conducted.

A. Suggested study protocol

A small formative study could start with a fixed set of representative tasks and ask each participant to think aloud while using a reset demo. Observers should record where participants hesitate, which labels they misunderstand, and whether they distinguish a request from a confirmed rental. The session should finish with questions about the estimated price, handoff, and absence of payment collection. Findings would guide interface changes; a claim about effectiveness would require a defined sample and analysis plan.

B. Technical extension path

If the concept proceeds beyond class demonstration, booking creation and status changes should move to server-side transactions. The server would re-evaluate date conflicts at commit time and authorize each action against an authenticated account. Shared storage would allow the owner and renter to see one canonical booking. This change would require a privacy model for student data, auditability of status changes, and a clear operational policy for disputes. It cannot be represented as a minor configuration change to the current localStorage build.

VIII. CONCLUSION

CampusLoop demonstrates a structured student rental flow through a responsive, browser-local application. The inspected tests verify selected date, ownership, transition, and listing rules. The work is best understood as a design and functional prototype. Its next step is empirical evaluation with students followed by a shared, authenticated implementation if the concept proves useful.

ACKNOWLEDGMENT

The authors thank Prof. Ifrah Sutar for project guidance and the Department of AI and Data Science at Dr. D. Y. Patil Institute of Technology, Pimpri, Pune.

REFERENCES

[1] R. Belk, "You are what you can access: Sharing and collaborative consumption online," Journal of Business Research, vol. 67, no. 8, pp. 1595-1600, 2014. doi:10.1016/j.jbusres.2013.10.001.

[2] J. Hamari, M. Sjoklint, and A. Ukkonen, "The sharing economy: Why people participate in collaborative consumption," Journal of the Association for Information Science and Technology, vol. 67, no. 9, pp. 2047-2059, 2016. doi:10.1002/asi.23552.

[3] United Nations, "Sustainable Development Goal 12: Responsible Consumption and Production," Target 12.5. https://sdgs.un.org/goals/goal12 (accessed 13 Oct. 2026).

[4] CampusLoop, project README and source code, local repository, inspected 14 Oct. 2026.

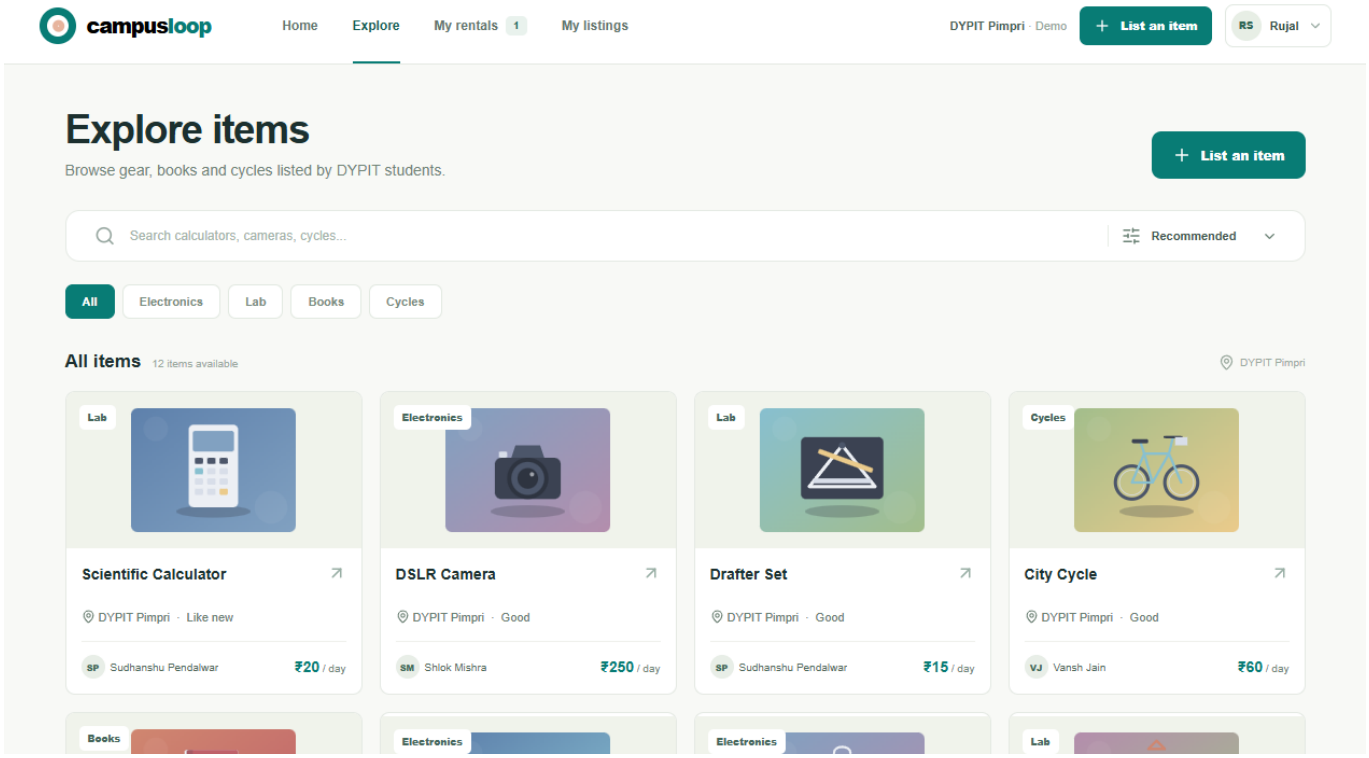


Fig. 2. Searchable sample catalog with category filters and sorting.

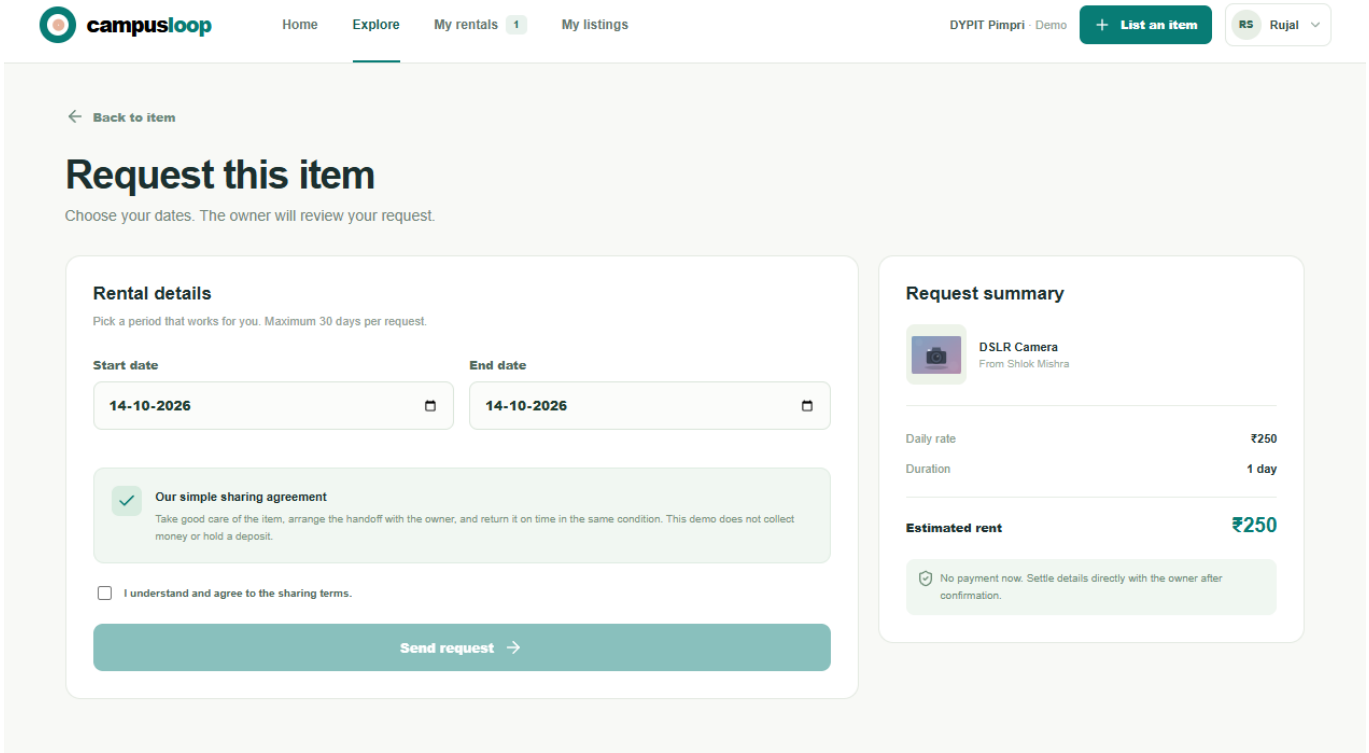


Fig. 3. Request form with date selection, sharing terms, and an estimated rent. No payment is collected.

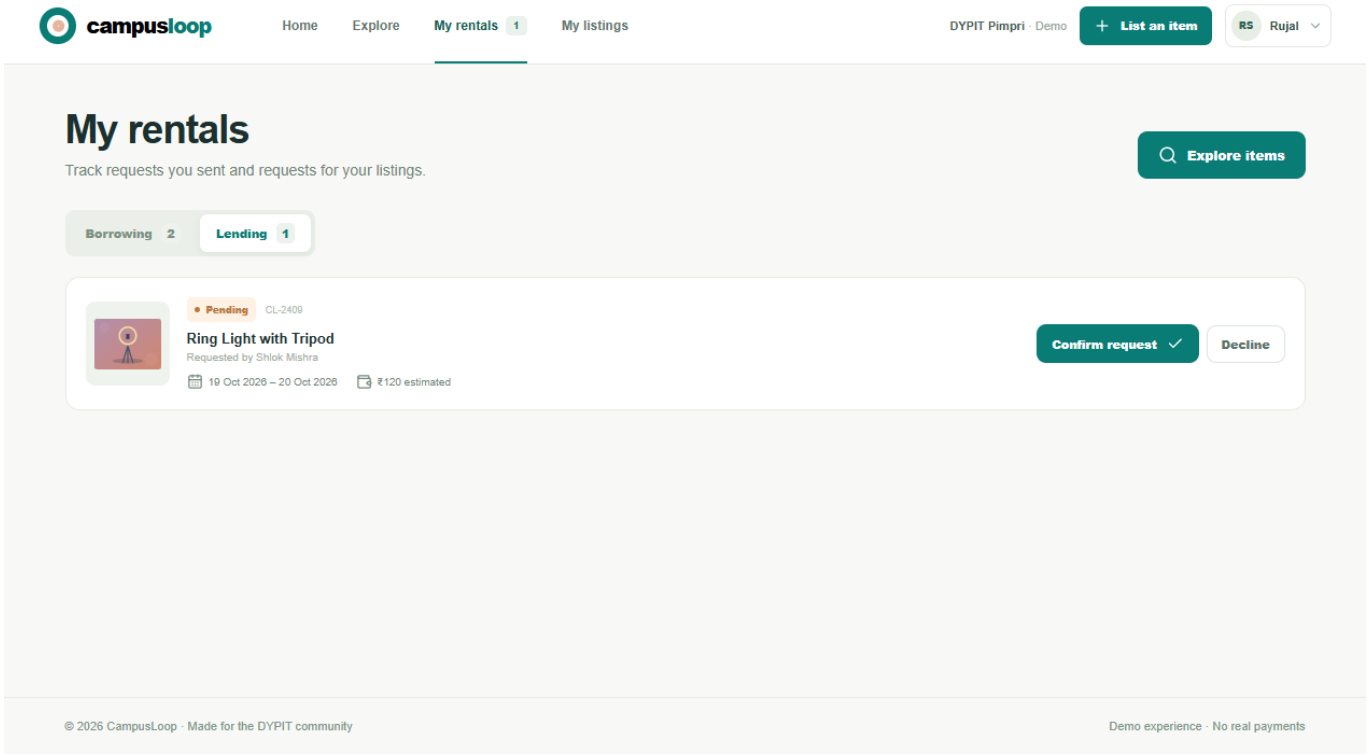


Fig. 4. Lending view displays a sample booking state.

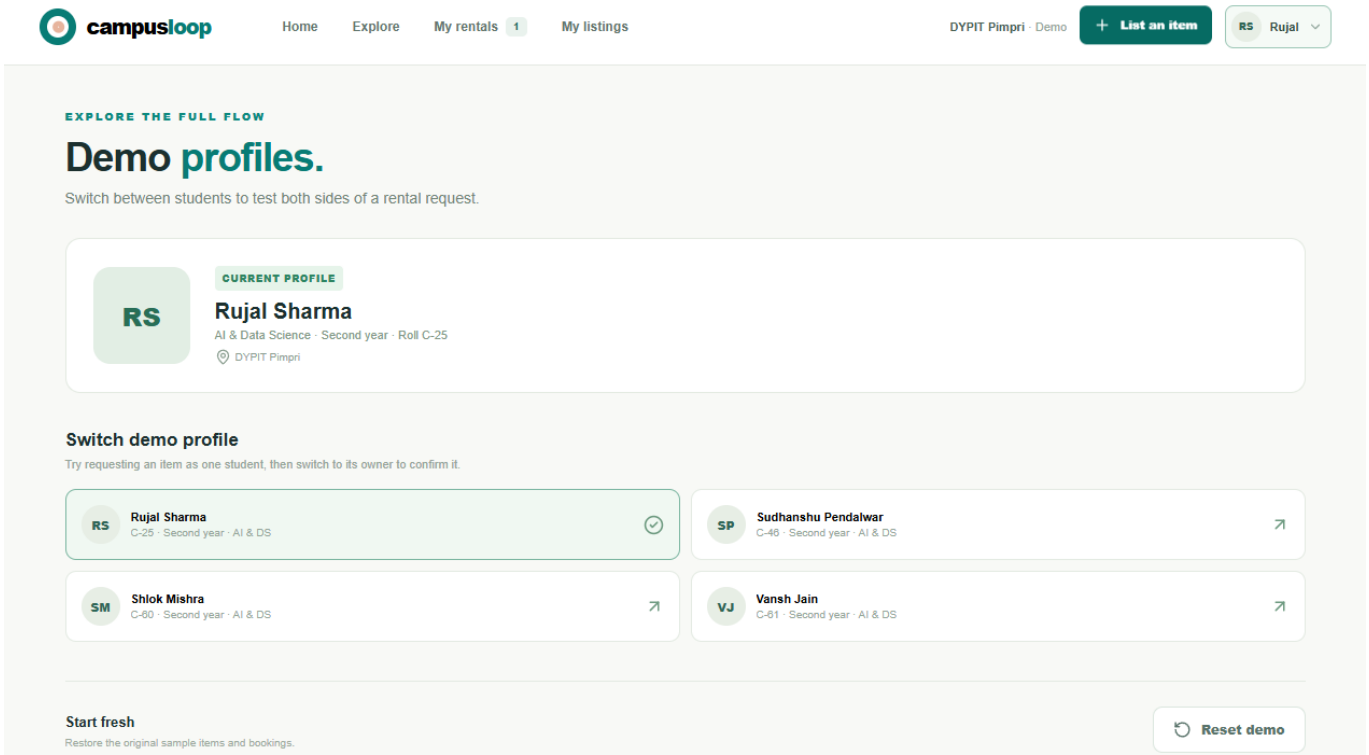


Fig. 5. Four switchable second-year AI and Data Science demo profiles.